

Karnatak University Dharwad
and
Karnatak University Mathematics Teachers
Association (KUMTA)

B.Sc. I Semester (NEP- Revised)
Laboratory Manual – 2024-25

Prepared by

Dr. Shashikant M.Sc., Ph.D.

Assistant Professor

Department of Mathematics

JSS Banashankari Arts, Commerce and
Shantikumar Gubbi Science College, Dharwad

and

Dr. Krishna Chavaraddi M.Sc., Ph.D.

Assistant Professor

Department of Mathematics

S. S. Govt. First Grade College and P. G. Centre, Naragund

Program No. 1: Program to compute addition and subtraction of matrices.

Program:

```
//Addition and subtraction of two matrices  
clc  
A=input("Enter the first matrix")  
B=input("Enter the second matrix")  
disp("The first matrix is")  
disp(A)  
disp("The second matrix is")  
disp(B)  
[m,n]=size(A);  
if size(A)==size(B) then  
    for i=1:m  
        for j=1:n  
            C(i,j)=A(i,j)+B(i,j);  
            D(i,j)=A(i,j)-B(i,j);  
        end;  
    end;  
    disp("Addition of two matrices is",C)  
    disp("Subtraction of two matrices is",D)  
else  
    disp("Addition and subtraction of given matrices are not defined")  
end
```

Output:

Enter the first matrix[1 8 -10;21 7 9;17 4 6;-4 28 19]

Enter the second matrix[22 5 6;12 -6 10;7 35 6;-16 44 21]

"The first matrix is"

1. 8. -10.
21. 7. 9.
17. 4. 6.
-4. 28. 19.

"The second matrix is"

22. 5. 6.
12. -6. 10.
7. 35. 6.
-16. 44. 21.

"Addition of two matrices is"

23. 13. -4.
33. 1. 19.
24. 39. 12.
-20. 72. 40.

"Subtraction of two matrices is"

-21. 3. -16.
9. 13. -1.
10. -31. 0.
12. -16. -2.

Program No. 2: Program to compute multiplication of matrices

Program:

```
//Multiplication of two matrices
clc
A=input("Enter the first matrix")
B=input("Enter the second matrix")
disp(A)
disp(B)
[m,n]=size(A);
[k,l]=size(B);
if n==k then
    C=A*B
    disp("Multiplication of two matrices is",C)
else
    disp("Matrices are of invalid order")
end
```

Output:

Enter the first matrix[10 2 3;4 8 -2;8 2 17]

Enter the second matrix[4 20 5;8 16 6;9 12 7]

10. 2. 3.

4. 8. -2.

8. 2. 17.

4. 20. 5.

8. 16. 6.

9. 12. 7.

"Multiplication of two matrices is"

83. 268. 83.

62. 184. 54.

201. 396. 171.

Program No. 3: Program to compute rank of matrix and row reduced echelon form

Program:

```
//Rank of a matrix and its row reduced echelon form  
clc  
A=input("Enter the matrix")  
disp(A)  
R=rank(A)  
B=rref(A)  
disp("Rank of a matrix",R)  
disp("Row reduced echelon form of a given matrix",B)
```

Output:

Enter the matrix[1 1 1 2;2 1 -3 -6;2 -3 1 2]

```
1. 1. 1. 2.  
2. 1. -3. -6.  
2. -3. 1. 2.
```

"Rank of a matrix"

```
3.
```

"Row reduced echelon form of a given matrix"

```
1. 0. 0. 0.  
0. 1. 0. 0.  
0. 0. 1. 2.
```

Program No. 4: Program to solve system of non-homogeneous linear equations

Program:

```
//System of non-homogeneous linear equations
clc
A=input("Enter the coefficient matrix A")
B=input("Enter the constant vector B")
[m,n]=size(A)
C=[A,B]
disp("The augmented matrix is")
disp(C)
r1=rank(A)
r2=rank(C)
disp("Rank of the coefficient matrix is",r1)
disp("Rank of the augmented matrix is",r2)
if (m>=n & r2<n) then
disp("System is consistent and has infinitely many solution")
else if (m>=n & r1==r2) then
disp("System is consistent and has unique solution")
X=A\B
disp("Solution of the system is", X)
else if(r1~=r2)
disp("System is inconsistent and has no solution")
else if(m<n)
disp("System has infinitely many solution")
end
end
end
end
```

Output 1:

Enter the coefficient matrix A[3 1 1;-1 1 -2;2 2 2;-2 2 -3]

Enter the constant vector B[8;-5;12;7]

"The augmented matrix is"

3. 1. 1. 8.
-1. 1. -2. -5.
2. 2. 2. 12.
-2. 2. -3. 7.

"Rank of the coefficient matrix is"

3.

"Rank of the augmented matrix is"

4.

"System is inconsistent and has no solution"

Output 2:

Enter the coefficient matrix A[3 1 1;-1 1 -2;2 2 2;-2 2 -3]

Enter the constant vector B[8;-5;12;-7]

"The augmented matrix is"

3. 1. 1. 8.
-1. 1. -2. -5.
2. 2. 2. 12.
-2. 2. -3. -7.

"Rank of the coefficient matrix is"

3.

"Rank of the augmented matrix is"

3.

"System is consistent and has unique solution"

"Solution of the system is"

1.0000000
2.0000000
3.0000000

Program No. 5: Program to check whether the given function is continuous at a point or not

Program 5(a) (For continuous functions):

```
// Program to check the continuity of a function at a point
clc
function y = step_function(x)
    if x ~= 3 then
        y = 10 - 2 * x;
    else
        y = x + 1;
    end
endfunction
function is_continuous(x0)
    h = 0.000001;
    tol = 0.00001;
    LHL = abs(step_function(x0 - h));
    RHL = abs(step_function(x0 + h));
    fx0 = abs(step_function(x0));
    disp("LHL = " + string(LHL));
    disp("RHL = " + string(RHL));
    disp("f(x0) = " + string(fx0));
    if abs(LHL - RHL) < tol & abs(LHL - fx0) < tol then
        disp("The function is continuous at x = " + string(x0));
    else
        disp("The function is not continuous at x = " + string(x0));
    end
endfunction
x0 = 3;
is_continuous(x0);
```

Output:

```
"LHL = 4.000002"
"RHL = 3.999998"
"f(x0) = 4"
"The function is continuous at x = 3"
```

Program -5(b) (For non-continuous functions):

```
// Program to check the continuity of a function at a point
clc
function y = step_function(x)
    if x ~= 0 then
        y=x^3+3;
    else
        y = 1;
    end
endfunction
function is_continuous(x0)
    h = 0.000001;
    tol=0.00001;
    LHL = abs(step_function(x0 - h));
    RHL = abs(step_function(x0 + h));
    fx0 = abs(step_function(x0));
    disp("LHL = " + string(LHL));
    disp("RHL = " + string(RHL));
    disp("f(x0) = " + string(fx0));
    if abs(LHL - RHL) < tol & abs(LHL - fx0) < tol then
        disp("The function is continuous at x = " + string(x0));
    else
        disp("The function is not continuous at x = " + string(x0));
    end
endfunction
x0=0;
is_continuous(x0);
```

Output:

```
"LHL = 3"
"RHL = 3"
"f(x0) = 1"
"The function is not continuous at x = 0"
```

Program No. 6: Program to solve example based on intermediate value theorem

Program:

```
//Program on Intermediate Value Theorem
```

```
clc
```

```
deff('y=f(x)','y=x^2-x-12')
```

```
a = input(" Enter the lower limit of interval")
```

```
b = input("Enter the upper limit of the interval")
```

```
if f(a)*f(b)>0 then
```

```
    disp("Change the interval")
```

```
else
```

```
    disp("Since f(a) & f(b) are off opposite signs, there should exist a point c between a & b  
such that f(c)=0 ")
```

```
    c = fsolve(a,f);
```

```
    disp('C='+string(c))
```

```
if a<c & c<b then
```

```
    disp("Intermediate value theorem is satisfied")
```

```
else
```

```
    disp("Intermediate value theorem is not satisfied")
```

```
end
```

```
end
```

Output:

Enter the lower limit of interval 3

Enter the upper limit of the interval 5

"Since f(a) & f(b) are off opposite signs, there should exist a point c between a & b such that
f(c)=0 "

"C=4"

"Intermediate value theorem is satisfied"

Program No. 7: Program to find the angle between the radius vector and tangent.

Program:

```
// Function to compute the derivative numerically
clc
function dr=numerical_derivative(f, theta, h)
    dr = (f(theta + h) - f(theta - h)) / (2 * h);
endfunction

// Function to calculate angle between radius vector and tangent
function angle=find_angle(r_func, theta, h)
    r = r_func(theta); // Calculate r
    dr_dtheta = numerical_derivative(r_func, theta, h); // Calculate dr/dtheta
    tan_theta = r / dr_dtheta; // Calculate tan(phi)
    angle = atan(tan_theta); // Calculate angle in radians
endfunction

// Define the polar function r = a(1 - cos(theta))
function r=polar_function(theta)
    a = 1; // Set the value of 'a' (can be adjusted)
    r = a * (1 - cos(theta));
endfunction

// Main program
theta = %pi / 4; // Example angle in radians
h = 1e-5; // Small step size for numerical differentiation

// Calculate angle
phi = find_angle(polar_function, theta, h);

// Display result
disp("Angle between radius vector and tangent (in radians): " + string(phi));
disp("Angle in degrees: " + string(phi * 180 / %pi));
```

Output:

"Angle between radius vector and tangent (in radians): 0.3926991"

"Angle in degrees: 22.5"

Program No. 8. Program to find the curvatures of the given curves.

Program:

```
// Function to calculate the curvature of the curve  $y^2 = 4ax$ 
clc
function curvature=calculate_curvature(a, x0)
    // Define the curve equation  $y = \sqrt{4 * a * x}$ 
    function y_val=y(x)
        y_val = sqrt(4 * a * x); // Curve equation
    endfunction

    // Numerical differentiation to find first derivative (dy/dx) using finite differences
    h = 1e-5; // Small step size for numerical differentiation
    dy_dx = (y(x0 + h) - y(x0 - h)) / (2 * h); // Central difference method

    // Numerical differentiation to find second derivative ( $d^2y/dx^2$ ) using finite differences
    d2y_dx2 = (y(x0 + h) - 2 * y(x0) + y(x0 - h)) / (h^2); // Central difference method

    // Curvature formula
    curvature = abs(d2y_dx2) / (1 + dy_dx^2)^(3/2);
endfunction

// Parameters
a = 1; // Parameter a
x = 1; // x-value

// Calculate curvature
curvature = calculate_curvature(a, x);

// Display result
disp("Curvature of the curve  $y^2 = 4ax$  at  $x = 1$  and  $a = 1$ :");
disp("Curvature: " + string(curvature));
```

Output:

```
"Curvature of the curve  $y^2 = 4ax$  at  $x = 1$  and  $a = 1$ :"
"Curvature: 0.1767767"
```

Program No. 9. Program to find the n^{th} derivative of e^{ax} , and trigonometric functions.

Program 1:

```
// Define the function f(x) = e^(a*x)
clc
function y=f(x, a)
    y = exp(a * x);
endfunction

// Function to calculate the n-th derivative using finite differences
function nth_derivative=finite_difference_nth_derivative(f, a, x, n, h)
    sum = 0;
    for k = 0:n
        // Calculate the central difference terms using appropriate offsets
        sum = sum + (-1)^(n - k) * (factorial(n) / (factorial(k) * factorial(n-k))) * f(x + (k - n/2)
* h, a);
    end
    nth_derivative = sum / (h^n);
endfunction

// Parameters
a = 2;    // Value of a
x = 1;    // Point at which we calculate the derivative
n = 3;    // Derivative order (3rd derivative in this case)
h = 0.01; // Step size for finite differences

// Calculate the nth derivative using finite differences
nth_derivative_value = finite_difference_nth_derivative(f, a, x, n, h);

// Display the finite difference result
disp("nth derivative of e^(a*x) at x=1: " + string(nth_derivative_value));
```

Output:

"nth derivative of e^(a*x) at x=1: 59.115404"

Program 2:

```

// Define the function  $f(x) = \sin(ax + b)$ 
clc
function y=f_sin(x, a, b)
    y = sin(a * x + b);
endfunction

// Function to calculate the n-th derivative using finite differences
function nth_derivative=finite_difference_nth_derivative(f, a, b, x, n, h)
    sum = 0;
    for k = 0:n
        // Calculate the central difference terms using appropriate offsets
        sum = sum + (-1)^(n - k) * (factorial(n) / (factorial(k) * factorial(n-k))) * f(x + (k - n/2)
            * h, a, b);
    end
    nth_derivative = sum / (h^n);
endfunction

// Parameters for sin
a = 2; // Value of a
b = 1; // Value of b
x = 1; // Point at which we calculate the derivative
n = 3; // Derivative order (3rd derivative in this case)
h = 0.01; // Step size for finite differences

// Calculate the nth derivative of  $\sin(ax + b)$  using finite differences
nth_derivative_value_sin = finite_difference_nth_derivative(f_sin, a, b, x, n, h);

// Display the result
disp("nth derivative of  $\sin(ax + b)$  at x=1: " + string(nth_derivative_value_sin));

```

Output:

"nth derivative of $\sin(ax + b)$ at x=1: 7.919544"

Program 3:

```

// Define the function  $f(x) = \cos(ax + b)$ 
clc
function y=f(x, a, b)
    y = cos(a * x + b);
endfunction

// Function to calculate the n-th derivative using finite differences
function nth_derivative=finite_difference_nth_derivative(f, a, b, x, n, h)
    sum = 0;
    for k = 0:n
        // Calculate the central difference terms using appropriate offsets
        sum = sum + (-1)^(n - k) * (factorial(n) / (factorial(k) * factorial(n-k))) * f(x + (k - n/2)
* h, a, b);
    end
    nth_derivative = sum / (h^n);
endfunction

// Parameters
a = 2; // Value of a
b = 1; // Value of b
x = 1; // Point at which we calculate the derivative
n = 3; // Derivative order (3rd derivative in this case)
h = 0.01; // Step size for finite differences

// Calculate the nth derivative using finite differences
nth_derivative_value = finite_difference_nth_derivative(f, a, b, x, n, h);

// Display the result
disp(nth_derivative_value);

```

Output:

1.1289036

Program No. 10. Program to find the n^{th} derivative of $e^{ax}\sin(bx + c)$ and $e^{ax}\cos(bx + c)$.

Program 1:

```
// Define the function f(x) = e^(a*x) * sin(b*x + c)
clc
function y=f(x, a, b, c)
    y = exp(a * x) * sin(b * x + c);
endfunction

// Function to calculate the n-th derivative using finite differences
function nth_derivative=finite_difference_nth_derivative(f, a, b, c, x, n, h)
    sum = 0;
    for k = 0:n
        // Calculate the central difference terms using appropriate offsets
        sum = sum + (-1)^(n - k) * (factorial(n) / (factorial(k) * factorial(n-k))) * f(x + (k - n/2) * h, a, b,
c);
    end
    nth_derivative = sum / (h^n);
endfunction

// Parameters
a = 2; // Value of a
b = 3; // Value of b
c = 1; // Value of c
x = 1; // Point at which we calculate the derivative
n = 3; // Derivative order (3rd derivative in this case)
h = 0.01; // Step size for finite differences

// Calculate the nth derivative using finite differences
nth_derivative_value = finite_difference_nth_derivative(f, a, b, c, x, n, h);

// Display the result
disp("nth derivative of e^(a*x) * sin(b*x + c) at x=1: " + string(nth_derivative_value));
```

Output:

"nth derivative of $e^{ax}\sin(bx + c)$ at x=1: 213.79381"

Program 2:

```

// Define the function  $f(x) = e^{(a*x)} * \cos(b*x + c)$ 
clc
function y=f(x, a, b, c)
    y = exp(a * x) * cos(b * x + c);
endfunction

// Function to calculate the n-th derivative using finite differences
function nth_derivative=finite_difference_nth_derivative(f, a, b, c, x, n, h)
    sum = 0;
    for k = 0:n
        // Calculate the central difference terms using appropriate offsets
        sum = sum + (-1)^(n - k) * (factorial(n) / (factorial(k) * factorial(n-k))) * f(x + (k - n/2)
* h, a, b, c);
    end
    nth_derivative = sum / (h^n);
endfunction

// Parameters
a = 2; // Value of a
b = 1; // Value of b
c = 0; // Value of c
x = 1; // Point at which we calculate the derivative
n = 3; // Derivative order (3rd derivative in this case)
h = 0.01; // Step size for finite differences

// Calculate the nth derivative using finite differences
nth_derivative_value = finite_difference_nth_derivative(f, a, b, c, x, n, h);

// Display the result
disp("nth derivative of  $e^{(a*x)} * \cos(b*x + c)$  at x=1: " + string(nth_derivative_value));

```

Output:

"nth derivative of $e^{(a*x)} * \cos(b*x + c)$ at x=1: -60.414874"

Program No. 11. Program to find the Taylor's and Maclaurin's expansions of the given functions.

Program:

// Function to calculate factorial

```
function result=fact(n)
    if n == 0 then
        result = 1;
    else
        result = prod(1:n);
    end
endfunction
```

// Function to calculate finite difference for the nth derivative

```
function derivative_value=finite_difference_derivative(f, a, x0, n, h)
    sum = 0;
    for k = 0:n
        sum = sum + (-1)^(n - k) * (fact(n) / (fact(k) * fact(n - k))) * f(x0 + (k - n / 2) * h, a);
    end
    derivative_value = sum / (h^n);
endfunction
```

// Function to compute the Taylor series expansion

```
function display_taylor_series_fd(f, a, x0, n_terms, h)
    disp("Taylor Series Expansion around x=" + string(x0) + ":");
    for k = 0:n_terms
        // Compute the k-th derivative using finite difference
        f_k = finite_difference_derivative(f, a, x0, k, h);
        // Add the k-th term to the series
        term = f_k / fact(k);
        if k == 0 then
            disp(string(term));
        else
            disp(string(term) + " * (x - " + string(x0) + ")^" + string(k));
        end
    end
endfunction
```

// Function to compute the Maclaurin series expansion (Taylor series at x0=0)

```
function display_maclaurin_series_fd(f, a, n_terms, h)
    disp("Maclaurin Series Expansion around x=0:");
    for k = 0:n_terms
        // Compute the k-th derivative using finite difference
        f_k = finite_difference_derivative(f, a, 0, k, h);
```

```

// Add the k-th term to the series
term = f_k / fact(k);
if k == 0 then
    disp(string(term));
else
    disp(string(term) + " * x^" + string(k));
end
end
endfunction

// Function definition: f(x) = e^(ax)
function y=example_function(x, a)
    y = exp(a * x);
endfunction

// Parameters
a = 1;    // Coefficient of x in the exponent
x0 = 1;   // Point of expansion for Taylor series
n_terms = 5; // Number of terms in the series
h = 0.01; // Step size for finite differences

// Display the Taylor and Maclaurin series
display_taylor_series fd(example_function, a, x0, n_terms, h); // Taylor series around x=1
display_maclaurin_series fd(example_function, a, n_terms, h); // Maclaurin series (x0=0)

```

Output:

"Taylor Series Expansion around x=1:"

"2.7182818"

"2.7182932 * (x - 1)^1"

"1.3591522 * (x - 1)^2"

"0.4530526 * (x - 1)^3"

"0.1132636 * (x - 1)^4"

"0.0226527 * (x - 1)^5"

"Maclaurin Series Expansion around x=0:"

"1"

"1.0000042 * x^1"

"0.5000042 * x^2"

"0.1666687 * x^3"

"0.0416674 * x^4"

"0.0083334 * x^5"

KUMT Association

Program No. 12. Program on tracing of standard curves in Cartesian form.

Program 1:

// 1. Plot the curve catenary in Cartesian form

```
x=[-5:0.01:5]
```

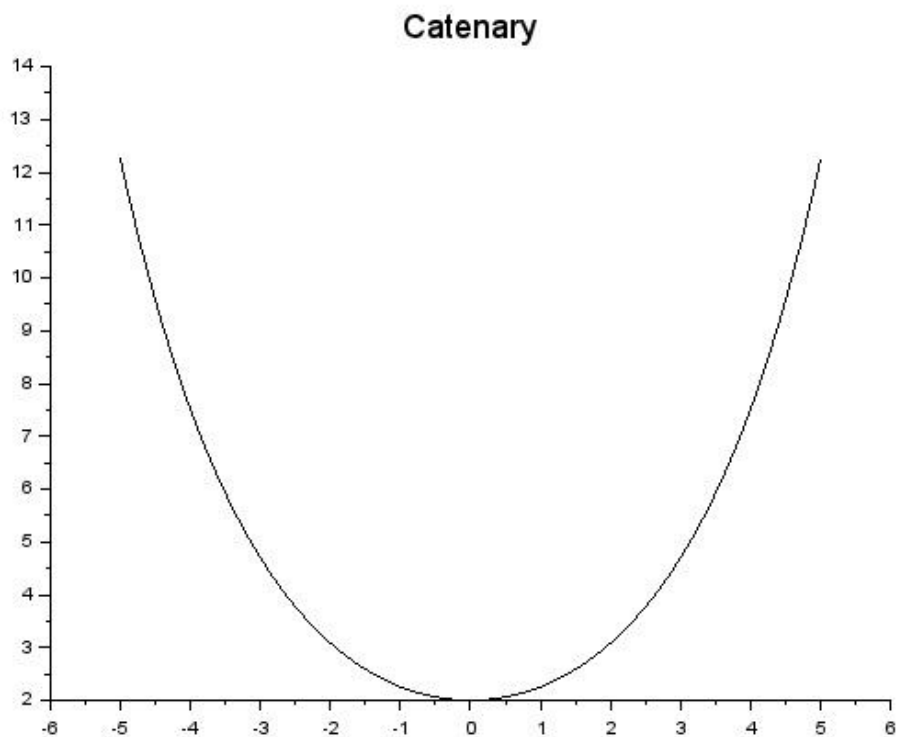
```
a=2;
```

```
y=a*cosh(x/a);
```

```
plot2d(x,y)
```

```
title("Catenary ", 'fontsize',4)
```

Output:



Program 2:

// 2 Plot the curve Cissoids in Cartesian form

```
x=[0:0.1:15.99]
```

```
a=16;
```

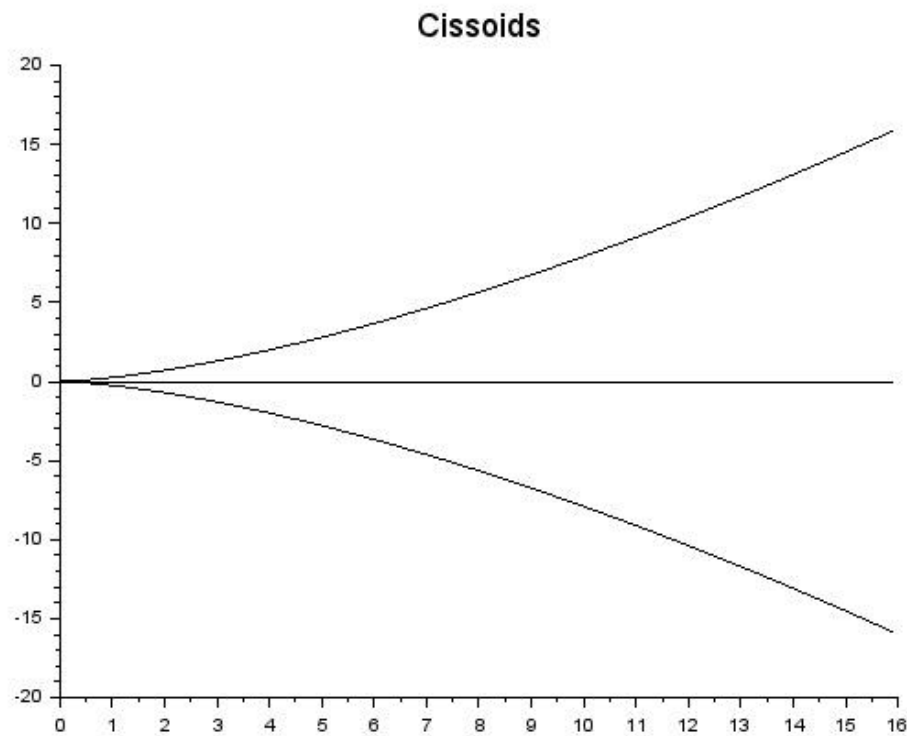
```
y=abs(sqrt((x^3)/(a-x))) ;
```

```
plot2d(x,y)
```

```
plot2d(x,-y)
```

```
title("Cissoids ", 'fontsize',4)
```

Output:



Program 3:

//3 Astroid and Strophoid in Cartesian form.

// Astroid : $y=(a^{2/3}-x^{2/3})^{3/2}$

`a=8;`

`x=[0:0.01:8]'`

`y=(a^(2/3)-(x^2)^(1/3))^(3/2);`

`x1=[-8:0.01:0]'`

`y1=(a^(2/3)-(x1^2)^(1/3))^(3/2);`

`b=gca()`

`b.x_location="origin"`

`b.y_location="origin"`

`plot2d(x,y)`

`plot2d(x,-y)`

`plot2d(x1,y1)`

`plot2d(x1,-y1)`

Output:

